

CSE 312

İşletim Sistemleri

7. Bölüm

Dosya Sistemleri

File Systems

Kapsamlı Konu Anlatımı & Çözümlü Sorular

Spring 2026

İÇİNDEKİLER

- 1** Dosya Sistemlerine Giriş
- 2** Tasarım Kararları (Design Decisions)
- 3** Dosya Yapıları ve Türleri
- 4** Metadata — Dosya Hakkındaki Veri
- 5** Alan Tahsisi (Space Allocation)
- 6** Dizinler (Directories)
- 7** Paylaşılan Dosyalar ve Bağlantılar (Links)
- 8** Journaling Dosya Sistemleri
- 9** Sanal Dosya Sistemi (VFS)
- 10** Disk Alanı Yönetimi
- 11** Disk Kotası (Disk Quotas)
- 12** Yedekleme (Backup)
- 13** Dosya Sistemi Tutarlılığı (Consistency)
- 14** Dosya Sistemi Performansı
- 15** Unix Dosya Sistemi — Detaylı İnceleme
- 16** Windows FAT Dosya Sistemi
- 17** ISO 9660 / CD-ROM Dosya Sistemi
- 18** Quiz Soruları ve Cevapları
- 19** Kitap Bölüm Sonu Soruları ve Çözümleri

1. Dosya Sistemlerine Giriş

Dosya sistemi (file system), işletim sisteminin en görünür parçalarından biridir. Kullanıcı deneyimini doğrudan etkiler ve yazılım ile donanım arasında temel bir soyutlama katmanı oluşturur.

Dosya (file): İşletim sisteminin depolama kaynakları için sunduğu mantıksal depolama birimidir. İki temel kullanım amacı vardır:

- Adres alanının uzantısı (geçici dosyalar — temporary files)
- Program sonlandıktan sonra da kalıcı olarak var olmaya devam eden bilgi deposu (persistent files)

Dizin (directory): Bir grup dosya için mantıksal 'kapsayıcı' görevi görür.

i Uzun Süreli Bilgi Depolamanın 3 Temel Gereksinimi

- Çok büyük miktarda bilgi depolanabilmeli
- Bilgi, o bilgiyi kullanan sürecin (process) sonlanmasından sonra da hayatta kalmalı
- Birden fazla süreç bilgiye eş zamanlı erişebilmeli

Bir diski sabit boyutlu bloklardan oluşan doğrusal bir dizi olarak düşünebilir ve bu blokları okuyup yazabiliriz. Bu temel model üç kritik soru doğurur:

Soru	Yanıt (Kısa)
Bilgiyi nasıl buluruz?	Dizin yapısı ve i-node tablosu ile
Bir kullanıcı diğerinin verilerini nasıl okuyamaz?	İzin (permission) sistemi ile
Hangi bloklar boş?	Serbest blok listesi (freelist) veya bitmap ile

Dosya sisteminin çözmesi gereken temel sorunlar

2. Tasarım Kararları (Design Decisions)

Bir dosya sistemi tasarlarken pek çok kritik karar alınması gerekir. Bu kararlar hem kullanıcı deneyimini hem de performansı derinden etkiler.

2.1 Dosya İsimlendirme (File Naming)

Konu	Seenekler / rnekler
Byk/Kk harf duyarlılıęı	Unix: duyarlı (README ≠ readme) Windows: duyarlı
Uzantı (extension)	Unix: isteęe baęlı, anlamsız; DOS: zorunlu 8.3 formatı
Karakter seti	ASCII / Unicode; bazı sistemler zel karakterlere izin verir
İsmlendirme dzeni	Tekdze (uniform) vs. hiyerarşik

Dosya ismlendirme tasarım boyutları

2.2 Farklı Sistemlerde Dosya Yolları

```

Unix : /this/is/a/unix.file.name
Windows : q:\Windows likes\m0n0c4SE
Multics : >Multics>usesup-a-level
OS/360 : os.360.looks.hierarchical.but.isnot(really)
VMS : vms-host::device[direct.ory.list]name.type;1

```

Farklı işletim sistemlerinde dosya yolu sözdizimi

2.3 Dięer Disk Srclerinin İsmlendirilmesi

İşletim Sistemi	Yöntem
Unix	mount komutu ile dosya sisteminin alt aęacı olarak baęlanır
Windows / VMS	Açıkça src harfi verilir (C:, D: gibi)
Multics	Tek aęaç; dizinler kalıcı disklerde yaęar
OS/360 / 370	Açık disk tanımlayıcı veya 'system catalog' (hiyerarşik yapı)

İşletim sistemlerinde disk adlandırma stratejileri

2.4 Ortam Özellikleri (Media Characteristics)

Ortam	Özellik
Modern diskler	512-byte sabit sektörler; OS'ye bölüm bilgisi açıkça sunulur
Eski mainframe diskler	Uygulama seçimli değişken-boyut sektörler
CD / DVD	Block-writable değil; sadece sıralı yazma
Flash disk	Bloklar belirli sayıda yazma sonrası aşınır (wear leveling gerekir)
WORM	Bloklar yazılabilir ve silinebilir ama üzerine yazılamaz

Farklı depolama ortamları ve özellikleri

2.5 Versiyonlama (Versioning)

Bazı dosya sistemleri (VMS, Tenex, TOPS-20) dosya adının bir parçası olarak sürüm numarası tutar (örn. report.txt;3). Plan 9 ise dosya sistemini geçmişte herhangi bir anda görüntülemenize olanak tanır.

i Tasarım Kararları Özeti

- Dosya adı politikası (büyük/küçük harf, uzantı, karakter seti)
- Hiyerarşik mi, yoksa düz (flat) yapı mı?
- Erişim kontrolü türü (permission, ACL, capability)
- Ortam özellikleri (disk, CD, flash, WORM)
- Hata kurtarma (journaling, fsck)
- Kayıt-, byte- veya blok-yapılı dosya mı?
- Versiyonlama desteği var mı?

3. Dosya Yapıları ve Türleri

Dosyalar fiziksel diskte nasıl organize edilir? Tarihsel olarak üç farklı yapı kullanılmıştır:

Yapı	Açıklama	Örnek
(a) Byte dizisi (Byte sequence)	Rastgele adreslenebilir byte'ların sırası. Uygulama kendi formatını tanımlar.	Unix, Windows
(b) Kayıt dizisi (Record sequence)	Sabit veya değişken boyutlu kayıtların (record) ardışık sırası. Eski sistemlerde kart veya satır imajları.	IBM OS/360
(c) Ağaç (Tree)	B-ağacı veya benzeri bir yapı; anahtar-tabanlı arama destekler.	IBM ana çerçeve veri tabanları

Üç temel dosya yapısı

Modern sistemlerin tercihi: Neredeyse tüm modern işletim sistemleri (Unix, Windows, macOS) byte dizisi modelini kullanır. Bu yaklaşım esneklik sağlar çünkü format yorumu uygulamaya bırakılır.

3.1 Dosya Türleri (File Types)

Bazı işletim sistemleri belirli dosyaları 'özel' olarak ele alır:

- Unix'te çoğu G/Ç aygıtının bir dosya adı vardır (/dev/sda, /dev/null)
- Dizinler (directories) bazen sadece dosyadır
- Bazı iletişim kanallarının (socket, pipe) dosya adı vardır

Dosya Türü	Açıklama
Regular file (düzenli dosya)	Veriden oluşan normal dosya
Directory (dizin)	Dosya adlarını i-node numaralarıyla eşleştiren özel dosya
Character special file	Terminaller, yazıcılar gibi karakter-tabanlı aygıtlar
Block special file	Diskler gibi blok-tabanlı aygıtlar
FIFO / Named pipe	Süreçler arası iletişim (IPC)
Symbolic link	Başka bir dosyaya işaret eden kısayol
Socket	Ağ veya yerel IPC için

Unix dosya türleri

4. Metadata — Dosya Hakkındaki Veri

Metadata, dosyanın içeriği dışındaki tüm bilgileri kapsar. En belirgin örnek disk bloklarının listesidir, ancak çok daha fazlası vardır:

Metadata Alanı	Açıklama
Disk blokları	Dosyanın hangi fiziksel bloklarda bulunduğu
Dosya sahibi (owner)	Dosyayı oluşturan kullanıcı (UID/GID)
Oluşturma / değiştirme / erişim zamanı	ctime, mtime, atime
İzinler (permissions)	rwX bitleri, ACL
Bağlantı sayısı (link count)	Kaç dizin girişi bu dosyayı gösteriyor
Dosya boyutu	Bayt cinsinden toplam boyut
Blok boyutu	I/O için tercih edilen blok büyüklüğü
Aygıt (device)	Dosyanın bulunduğu aygıt numarası

Linux struct stat alanları

4.1 Linux struct stat

```

struct stat {
    dev_t st_dev; /* aygıt (device) */
    ino_t st_ino; /* i-node numarası */
    mode_t st_mode; /* koruma / izinler */
    nlink_t st_nlink; /* hard link sayısı */
    uid_t st_uid; /* sahibin kullanıcı ID'si */
    gid_t st_gid; /* sahibin grup ID'si */
    dev_t st_rdev; /* aygıt türü (inode aygıtısa) */
    off_t st_size; /* byte cinsinden toplam boyut */
    blksize_t st_blksize; /* dosya sistemi I/O blok boyutu */
    blkcnt_t st_blocks; /* tahsis edilen 512-byte blok sayısı */
    time_t st_atime; /* son erişim zamanı */
    time_t st_mtime; /* son değiştirme zamanı */
    time_t st_ctime; /* son durum değişim zamanı */
};

```

Linux'ta struct stat — dosya meta verisi yapısı

4.2 Metadata Nerede Saklanır?

Konum	Avantaj	Dezavantaj
Dosyanın kendisinde	Apple resource fork yaklaşımı; basit	Taşınabilir değil; kopyalamada kaybolur
Dizin girişinde	Metadata erişimi hızlı	Hard link'lerde kopyalar tutarlı tutulmalı; dizin şişer
Ayrı yapıda (i-node)	En esnek; hard link kolay	Ekstra disk erişimi gerektirir

Metadata depolama stratejilerinin karşılaştırması

5. Alan Tahsisi (Space Allocation)

Dosya sisteminin temel görevlerinden biri, dosyalara disk alanı tahsis etmektir. Üç ana strateji vardır:

5.1 Ardışık Tahsis (Contiguous Allocation)

Her dosya diskte ardışık (bitişik) bloklar kaplar. Dizin girişinde yalnızca başlangıç blok numarası ve uzunluk saklanır.

⚠ Ardışık Tahsis

- Avantaj: Çok basit; başlangıç + uzunluk yeterli
- Avantaj: Sıralı ve rastgele erişimde hızlı
- Dezavantaj: Dış parçalanma (external fragmentation)
- Dezavantaj: Dosya büyütmede esneksizlik — bitişik yer bulunamayabilir

5.2 Bağlantılı Tahsis (Linked Allocation)

Her dosya disk bloklarının bir bağlı listesidir. Her blok bir sonraki blok işaretçisi içerir. Dizin girişi yalnızca ilk bloğun adresini tutar.

⚠ Bağlantılı Tahsis

- Avantaj: Alana verimli; dış parçalanma yok
- Avantaj: Dosya büyütme kolay (sona ekle)
- Dezavantaj: Rastgele erişim yavaş — n. bloğa ulaşmak için n adım gerekir
- Dezavantaj: Bir blok bozulursa tüm zincir kopabilir

5.3 FAT — Bellekte Tablo ile Bağlantılı Tahsis

FAT (File Allocation Table), bağlı listenin işaretçilerini blokların içinden çıkarıp bellekte tutulan ayrı bir tabloya taşır. Bu sayede disk erişimi olmadan zincir takip edilebilir.


```
Örnek – test dosyası: blok 217 -> 618 -> 618 -> -1 (son)
```

```
FAT Tablosu (kısmi):
```

```
Blok | Sonraki
```

```
-----|-----
```

```
217 | 618
```

```
618 | -1 (EOF)
```

```
Dizin Girişi: test ... 217 (ilk blok)
```

FAT tablosu örneği — test dosyası 217 ve 618 numaralı bloklarda

⚠ FAT Analizi

- □ Disk erişimi gerekmeden zincir takip edilebilir
- □ Büyük disk için çok fazla bellek gerekir
- Örnek: 200 GB disk, 1 KB blok → 200M FAT girişi → ~800 MB RAM!

5.4 İndeksli Tahsis — I-node

Tüm işaretçiler tek bir yapıda — i-node (index node) — toplanır. I-node, dosyaya ait tüm metadata'yı ve disk bloklarının adreslerini içerir. Yalnızca açık dosyalar için belleğe alınır, diğerleri diskte tutulur.

□ I-node Avantajları

- Dış parçalanma yok (external fragmentation free)
- Rastgele erişim hızlı — doğrudan blok adresine atla
- Büyük dosyalar için çok kademeli dolaylı adresleme (indirect blocks)
- Küçük dosyalar için az overhead

5.5 Çok Kademeli Dolaylı Adresleme (Multi-level Indirection)

Unix V7 gibi klasik sistemlerde i-node yapısı şöyledir: 10 doğrudan blok adresi, ardından tek dolaylı (single indirect), çift dolaylı (double indirect) ve üçlü dolaylı (triple indirect) bloklara işaretçiler.

Seviye	Açıklama	Max Boyut (4KB blok, 4B ptr)
Doğrudan (Direct)	10 blok × 4 KB	40 KB
Tek dolaylı (Single indirect)	1024 × 4 KB	4 MB
Çift dolaylı (Double indirect)	1024 × 1024 × 4 KB	4 GB
Üçlü dolaylı (Triple indirect)	1024 ³ × 4 KB	~4 TB

Kademeli dolaylı adresleme ile maksimum dosya boyutu

Örnek hesaplama (ders notlarından): 10 doğrudan + 1 single indirect ile maksimum dosya boyutu: $4096 \times 10 + (4096/4) \times 4096 = 4.235.264 \text{ byte} \approx 4 \text{ MB}$

```
# Python ile max dosya boyutu hesabı:
block_size = 4096 # 4 KB
ptr_size = 4 # 4 byte pointer
ptrs_per_block = block_size // ptr_size # = 1024

direct = 10 * block_size
single_indir = ptrs_per_block * block_size
double_indir = ptrs_per_block**2 * block_size
triple_indir = ptrs_per_block**3 * block_size

total = direct + single_indir + double_indir + triple_indir
print(f'Max dosya boyutu: {total:,} byte')
# Çıktı: Max dosya boyutu: 4,299,161,600 byte ≈ 4 GB
```

I-node max dosya boyutu hesabı

Tahsis Yöntemi	Sıralı Erişim	Rastgele Erişim	Alan Verimliliği	Büyüme
Ardışık (Contiguous)	□□□	□□□	□□ (parçalanma)	□ Zor
Bağlantılı (Linked)	□□	□ (yavaş)	□□□	□ Kolay
FAT	□□	□□	□□	□ Kolay
I-node (İndeksli)	□□□	□□□	□□□	□ Kolay

Tahsis yöntemlerinin karşılaştırması

6. Dizinler (Directories)

Dizin, dosya adlarını i-node numaralarıyla eşleştiren özel bir dosyadır. Tek katlı (flat) yapıdan hiyerarşik ağaca doğru evrilmiştir.

6.1 Dizin Operasyonları

Sistem Çağrısı	Açıklama
create	Yeni dizin oluştur
delete	Dizini sil (önce boşaltılmalı)
opendir	Dizini okumak için aç
closedir	Dizini kapat
readdir	Sonraki dizin girişini oku
rename	Dizini yeniden adlandır
link	Bir dosyaya yeni bağlantı ekle
unlink	Dizin girişini kaldır

Dizin yönetimi sistem çağrıları

6.2 Dizin Girişi Yapısı

Yaklaşım	İçerik	Kullanım
Sabit boyutlu giriş (a)	Dosya adı + disk adresleri + öznitelikler	Eski sistemler (FAT benzeri)
I-node referanslı giriş (b)	Dosya adı + i-node numarası (öznitelikler i-node'da)	Modern Unix / Linux

İki farklı dizin giriş yaklaşımı

6.3 Uzun Dosya Adları

Yöntem	Açıklama
In-line (satır içi)	Ad direkt dizin girişine yazılır; değişken uzunluk
Heap (yığın)	Sabit boyutlu dizin girişi ada bir işaretçi tutar; ad ayrı bir 'heap' alanında saklanır

Uzun dosya adı yönetimi

6.4 . ve .. Girişleri

Unix'te her dizinde iki özel giriş bulunur: . mevcut dizinin i-node numarasına, .. üst dizinin i-node numarasına işaret eder. Bu sayede görelî yol gezintisi mümkündür.

⚠ Önemli Not

- . ve .. standart olarak bulunmakla birlikte, POSIX tarafından garantilenmez
- Yol arama döngü içermez; ağaç yapısı sağlamak için özel önlem alınır
- Symbolic link'ler döngü yaratabilir — bu yüzden takip sayısı sınırlandırılır

7. Paylaşılan Dosyalar ve Bağlantılar (Links)

Birden fazla kullanıcı veya izin aynı dosyayı paylaşmak isteyebilir. Unix iki tür bağlantı mekanizması sunar:

Özellik	Hard Link	Symbolic Link (Soft Link)
Ne işaret eder?	Doğrudan i-node numarasına	Başka bir yol adına (path)
Farklı dosya sistemi?	❑ Hayır — aynı FS olmalı	❑ Evet
Dizinlere?	❑ Genellikle yasak	❑ Mümkün
Orijinal silinirse?	Dosya hâlâ erişilebilir (link count > 0)	Bağlantı geçersiz olur (dangling)
Link count?	Her hard link +1 yapar	Etkilemez
stat() çağrısı	Gerçek dosyayı gösterir	Sembolik linkin kendisini gösterir (lstat)

Hard Link ile Symbolic Link karşılaştırması

```
# Hard link oluştur:
ln dosya.txt hard_link.txt

# Symbolic link oluştur:
ln -s dosya.txt soft_link.txt

# Link sayısını gör:
ls -l dosya.txt

# -rw-r--r-- 2 kullanıcı grup 1024 May 1 dosya.txt
# ^--- link count = 2
```

Hard link ve symbolic link kullanım örnekleri

7.1 Dosya Silme Senaryosu

Adım	Durum
(a) Başlangıç	A ve B dizinleri aynı dosyayı (hard link) gösteriyor; link count = 2
(b) Link oluşturuldu	Link oluşturulduktan sonra her iki dizin de i-node'u gösteriyor
(c) Orijinal sahibi siler	A dizininden silindi; link count = 1; B hâlâ erişebilir

Hard link ile dosya paylaşım ve silme senaryosu

8. Journaling Dosya Sistemleri

Sistem çökmesi durumunda dosya sistemi tutarsız bir durumda kalabilir. Klasik çözüm: fsck ile önyüklemede tutarlılık kontrolü. Modern çözüm: Journaling (günlükleme).

8.1 Unix'te Dosya Silme Operasyonu

Bir dosyayı silmek için 3 yazma operasyonu gerekir:

- Dosyayı dizininden kaldır
- I-node'u serbest i-node havuzuna geri ver
- Tüm disk bloklarını serbest blok havuzuna geri ver

⚠ Çökme Riski

- Eğer bu 3 yazma tamamlanmadan sistem çökerse tutarsızlık oluşur
- Yazmaların sıralanması kritik: disk her zaman güvenli bir durumda olmalı
- Örnek: Dizin girişini silmeden önce i-node'u yazmayın

8.2 Journaling Nasıl Çalışır?

Verilerin üzerine yazmak yerine değişiklikler önce bir journal alanına (günlük) eklenir. Dosya sistemi her zaman tutarlıdır çünkü yazmalar doğru sırada yapılır.

Journaling adımları:

1. Değişikliği journal'a yaz (commit kaydı)
2. Gerçek veriyi diske yaz
3. Journal girişini 'tamamlandı' olarak işaretle

Çökme sonrası kurtarma:

- Journal'da tamamlanmamış işlem varsa → geri al (rollback) veya yeniden yaz
- Journal temizse → dosya sistemi tutarlı

Journaling mekanizması

8.3 Modern Disklerle İlgili Uyarı

Modern diskler büyük yazma önbellekleri (2-16 MB) kullanır ve performans için yazmaları yeniden sıralar. Güç kesilirse tamponda bekleyen veriler kaybolabilir. Bu yüzden journaling tek başına yeterli olmayabilir — write barriers gerekebilir.

9. Sanal Dosya Sistemi (VFS — Virtual File System)

Tüm modern işletim sistemleri birden fazla dosya sistemi türünü destekler. VFS, bu farklı dosya sistemlerini tek tip bir arayüz arkasında gizler.

Katman	Rol
Kullanıcı süreci	open(), read(), write() sistem çağrıları
VFS katmanı	Tek tip arayüz; hangi FS olduğunu bilmez
Somut FS sürücüsü (ext4, FAT, NTFS...)	Gerçek disk operasyonlarını gerçekleştirir
Buffer cache	Blok önbellekleme
Aygıt sürücüsü	Donanıma doğrudan erişim

VFS katmanlı mimarisi

VFS iki temel soyutlama kullanır:

- vnode (sanal i-node): Her açık dosyayı temsil eden sanal nesne; gerçek i-node'a veya NFS'e işaret edebilir
- Süper blok (superblock): Bağlı her dosya sistemi için bir süper blok nesnesi tutulur

10. Disk Alanı Yönetimi

10.1 Blok Boyutu Seçimi

Blok boyutu dosya sisteminin temel parametresidir. Büyük blok → yüksek veri aktarım hızı, ama küçük dosyalar için israf. Küçük blok → verimli alan kullanımı, ama daha fazla I/O.

i Blok Boyutu Dengesi

- 4 KB blok: Çoğu modern sistemde standart tercih
- Büyük blok boyutu → Veri hızı artar (sol eksen), alan verimliliği düşer (sağ eksen)
- Küçük dosyalar (< 4 KB) için küçük blok daha verimli
- Dosyaların büyük çoğunluğu küçüktür — bu denge önemlidir

10.2 Serbest Blok Takibi

Yöntem	Açıklama	Avantaj / Dezavantaj
Bağlı liste (Linked list)	Serbest bloklar bir zincir oluşturur; ilk boş bloğa işaretçi saklanır	□ Basit □ Parçalı erişimde yavaş
Bitmap	Her blok için 1 bit: 1=dolu, 0=boş	□ Hızlı, kompakt □ Büyük disklerde bitmap büyür

Serbest blok izleme yöntemleri

Pratikte: Tam bir blok dolusu işaretçi bellekte tutulur, geri kalanlar diskte. Blok serbest bırakıldığında: eğer bellekteki blok dolu değilse oraya ekle, doluysa ayrı bir strateji uygula.

10.3 Disk Kotası (Disk Quotas)

Çok kullanıcıli sistemlerde her kullanıcının ne kadar disk alanı kullanabileceği sınırlandırılır. Her kullanıcı için kullanıcı başına kota tablosu tutulur:

Alan	Açıklama
Soft limit	Uyarı eşiği; kısa süre aşılabılır
Hard limit	Kesinlikle aşılamaz
Grace period	Soft limit aşıldıktan sonra sağlanan süre

Disk kotası parametreleri

11. Yedekleme (Backup)

Yedekleme iki temel nedenden yapılır:

- Felaketten kurtarma (recover from disaster): Disk arızası, yangın, sel
- Aptallıktan kurtarma (recover from stupidity): Kazara silinen dosyalar

11.1 Fiziksel Dump vs. Mantıksal Dump

Özellik	Fiziksel Dump (Physical)	Mantıksal Dump (Logical)
Ne yedekler?	Blok 0'dan itibaren tüm disk	Seçili dizinlerden başlayarak özyinelemeli
Hız	Hızlı	Daha yavaş
Boş bloklar?	Evet — boş bloklar da yedeklenir	Hayır — sadece gerçek dosyalar
Kötü bloklar?	Risk var	Atlanır
Artımlı yedek mümkün mü?	Hayır	Evet
Farklı disk geometrisine geri yükleme	Hayır	Evet

Fiziksel vs. mantıksal dump karşılaştırması

11.2 Mantıksal Dump Algoritması (4 Aşama)

Her i-node için birkaç bit içeren bir bitmap kullanılır. Algoritma 4 aşamada çalışır:

Aşama	İşlem	Amaç
1. Aşama	Değiştirilmiş tüm dosyaları ve her dizini işaretle	Değişen içeriği bul
2. Aşama	Altında değiştirilmiş dosya olmayan dizinlerin işaretini kaldır	Gereksiz dizinleri eleme
3. Aşama	İşaretili tüm dizinleri i-node sırasıyla dök	Dizin yapısını yedekle
4. Aşama	İşaretili tüm dosyaları dök	Dosya içeriklerini yedekle

Mantıksal dump algoritmasının 4 aşaması

11.3 Kurtarma Algoritması

- Diskte boş bir dosya sistemi oluştur
- En son tam yedekten geri yükle (önce dizinler, sonra dosyalar)
- Her artımlı yedek için aynı işlemi tekrarla

⚠ Kurtarma Sorunları

- Serbest bloklar: Kurtarma tamamlandıktan sonra yeniden oluşturulur
- Hard link'ler: İki dizinde bağlantılı dosya yalnızca bir kez kurtarılmalı

11.4 Unix Yedekleme Seviyeleri

Unix'te 0-9 arası yedek seviyeleri vardır:

Seviye	Açıklama
0	Tam yedek (everything)
1	Son seviye-1 yedekten bu yana değişen her şey
2	Son seviye-1 yedekten bu yana değişen her şey (seviye-1 ile aynı mantık)

Unix yedek seviyeleri

12. Dosya Sistemi Tutarlılığı (Consistency)

Sistem çökmesi durumunda dosya sistemi tutarsız bir durumda kalabilir. fsck (Unix) ve scandisk (Windows) önyüklemesi sırasında bunu tespit edip düzeltir.

12.1 Blok Tutarlılığı

fsck her blok için iki sayaç tutar:

- 1. Tablo: Bu blok kaç dosyada kullanılıyor?
- 2. Tablo: Bu blok serbest listede kaç kez var?

Durum	1. Sayaç	2. Sayaç	Sorun / Çözüm
(a) Tutarlı	1	1	Normal — her blok ya bir dosyada ya serbest listede
(b) Kayıp blok	0	0	Blok ne dosyada ne listede → serbest listeye ekle
(c) Serbest listede çift	0	2	Blok iki kez serbest listede → listeyi yeniden oluştur
(d) Veri bloğu çift	2	0	İki dosya aynı bloğu paylaşıyor → bloku kopyala, her birine ver

Olası blok tutarsızlığı durumları ve çözümleri

13. Dosya Sistemi Performansı

Disk erişimi bellek erişiminden çok daha yavaştır. Performansı artırmak için üç temel teknik kullanılır:

13.1 Önbellekleme (Caching — Buffer Cache)

Sık erişilen bloklar bellekte tutulur. Saf LRU yerine iki faktör göz önünde bulundurulur:

- Bu blok yakın zamanda yeniden kullanılacak mı?
- Bu blok dosya sisteminin tutarlılığı için kritik mi? (i-node blokları gibi)

⚠ **Önbellekleme Uyarısı**

- Güç kesilirse tampondaki değişiklikler kaybolur
- `sync()` veya `fsync()` çağrısı ile tampon diske boşaltılır
- I-node blokları gibi kritik bloklar MRU yerine LRU önceliğiyle yazılmalı

13.2 Önceden Okuma (Read-Ahead)

Erişim sıralı görünüyorsa, sonraki bloklar beklenmeksizin arka planda okunmaya başlanır. Bu sayede okuma gecikmesi örtülür.

Okuma adımları (Unix):

1. Mevcut byte offsetini blok numarasına çevir
2. 0 bloğu diskten oku
3. Doğru byte'ları kullanıcıya aktar
4. Mevcut byte offsetini güncelle
5. (İsteğe bağlı) Sıralı erişim görünüyorsa sonraki bloğu başlat (read-ahead)

Unix dosya okuma algoritması

13.3 Disk Kol Hareketi Azaltma (Cylinder Groups)

Klasik Unix'te i-node listesi diskin başında, veriler sonunda yer alırdı — her erişim uzun bir disk kolu hareketi gerektirirdi. Modern BSD Unix'te cylinder group (silindir grubu) yapısı kullanılır:

- Disk silindir gruplarına bölünür
- Her grup kendi i-node listesi, kendi serbest listesi ve kendi veri alanına sahiptir
- Yeni dosyalar mümkünse aynı cylinder group içinde oluşturulur — referans yerselliği

14. Unix Dosya Sistemi — Detaylı İnceleme

14.1 Dosya Sistemi Bileşenleri

Bileşen	Açıklama
Boot record (MBR)	Disk başında; BIOS tarafından önyüklemeye için kullanılır; disk bölüm tablosu da burada
Superblock	Dosya sistemi parametreleri: i-list konumu, temiz kapanma bayrağı, blok boyutu...
I-list	I-node dizisi; i-node numarası bu diziye indekstir
Freelist	Serbest blok listesi veya bitmap
Data area	Gerçek dosya verileri

Unix dosya sistemi disk yapısı bileşenleri

14.2 Dosya Açma (open())

open() adımları:

1. I-node gerekiyorsa diskten belleğe oku
2. I-node referans sayısını artır (reference count++)
3. Bir dosya tablosu (file table) girişi oluştur
4. Dosya tablosundaki indeksi uygulamaya file descriptor olarak ver

Tüm OS'lerde bu kavram 'file handle' veya 'file descriptor' olarak geçer

open() sistem çağrısı akışı

14.3 Dosya Oluşturma (create())

Mevcut dizin girişi var mı?

EVET → Dosyayı aç (gerekirse truncate)

HAYIR → Yeni giriş oluştur:

- Dizin yazma (normal dosya yazımı gibi)
- Serbest bir i-node bul ve tahsis et
- Dizin girişi serbest: i-node numarası = 0
- I-node serbest: link count = 0

create() sistem çağrısı akışı

14.4 Dosya Yazma (write())

```
Bir bloğun ortasına yazıyor muyuz?
EVET → 0 bloğu önce diskten oku
HAYIR → Serbest listeden yeni blok al, i-node'a ekle

Kullanıcıdan gelen veriyi tampona kopyala
Tamponu 'kirli' (dirty) olarak işaretle → bir süre sonra diske yazılacak
Dosya offset işaretçisini güncelle
```

write() sistem çağrısı akışı

14.5 Seek, Close, Link, Unlink

Operasyon	Ne Yapar?
seek()	Mevcut byte offsetini değiştirir — disk kolunu HAREKET ETTİRMEZ
close()	I-node referans sayısını azaltır; dosya tablosu girişini siler
link()	Yeni dizin girişi oluşturur; aynı i-node numarası; link count++
unlink()	Link count'u azaltır; sıfırsa ve ref count da sıfırsa dosya silinir

Temel dosya operasyonları özeti

⚠️ Önemli: Açık Dosya Silme

- unlink() açık bir dosyaya uygulanabilir — dosya hemen silinmez
- Son close() çağrısında silinir (link count = 0 VE ref count = 0)
- Sistem çökmesi bu durumda: i-node 'orphan' (kimsesiz) kalabilir — fsck temizler

14.6 /usr/ast/mbox Dosya Yolu Arama

Adım	İşlem
1	Kök dizinin i-node'unu oku (inode 1 veya 2)
2	Kök dizinde 'usr' adını ara → i-node numarasını bul
3	usr dizininin i-node'unu oku
4	usr dizininde 'ast' adını ara → i-node numarasını bul
5	ast dizininin i-node'unu oku
6	ast dizininde 'mbox' adını ara → i-node numarasını bul
7	mbox'ın i-node'unu oku → dosyaya eriş

/usr/ast/mbox dosya yolu arama adımları

15. Windows FAT Dosya Sistemi

Özellik	Açıklama
Kök dizin	Sınırlı boyut — sabit sayıda giriş
Dosya adı	8.3 formatı (8 karakter ad + 3 karakter uzantı)
Metadata konumu	Dizin girişinde (i-node yok)
FAT giriş boyutu	FAT12 / FAT16 / FAT32 — adres alanı genişliği
Max blok adresi	28-bit (FAT32) — max dosya boyutu 4 GB

Windows FAT dosya sistemi özellikleri

15.1 Uzun Dosya Adı Desteği (Windows 98+)

Geriye dönük uyumluluk koruyarak uzun ad desteği eklendi:

- Uzun adın tanınabilir kısmından 8.3 versiyonu türetilir
- Gerçek girişten önce sahte/geçersiz dizin girişleri eklenir
- DOS bu girişleri geçersiz kabul ederek atlar
- Kısa ad versiyonu silinirse sahtecilik anlaşılmasın diye checksum eklenir

16. ISO 9660 / CD-ROM Dosya Sistemi

CD-ROM dosya sistemi standartı ISO 9660, salt okunur medya için tasarlanmıştır.

Bileşen	Açıklama
İlk 16 blok	OS veya diğer amaçlar için ayrılmış
Primary volume descriptor	CD hakkında genel bilgi (yayıncı, oluşturma tarihi vb.)
Dizin girişi	Sabit uzunluklu standart alanlar + uzantı alanları

ISO 9660 CD-ROM disk yapısı

Rock Ridge Uzantıları

Uzantı	Açıklama
PX	POSIX öznitelikleri (izinler, owner, group)
PN	Aygıt major/minor numaraları
SL	Symbolic link
NM	Alternatif (uzun) dosya adı
TF	Zaman damgaları (ctime, mtime, atime)

Rock Ridge uzantı alanları

Joliet Uzantıları

- Uzun dosya adları (255 karaktere kadar)
- Unicode karakter seti desteği
- 8 seviyeden derin dizin yuvalama
- Uzantılı dizin adları

17. Quiz Soruları ve Cevapları

i Quiz Hakkında

- Aşağıdaki sorular sınav formatındadır — çoktan seçmeli değil, açık uçlu
- Her sorunun altında detaylı cevap verilmektedir
- Bu sorular gerçek sınavlarda çıkma olasılığı yüksek konuları kapsar

S1. Bir i-node'un 10 doğrudan blok adresi ve 1 single indirect blok adresi olduğunu varsayın. Blok boyutu 4 KB ve pointer boyutu 4 byte ise maksimum dosya boyutunu hesaplayın.

Doğrudan bloklar: $10 \times 4096 = 40,960$ byte

Single indirect blok, $4096/4 = 1024$ pointer içerir

Single indirect: $1024 \times 4096 = 4,194,304$ byte

Toplam: $40,960 + 4,194,304 = 4,235,264$ byte ≈ 4.04 MB

Not: Double indirect eklenseydi $\rightarrow +1024^2 \times 4096 \approx 4$ GB daha

S2. Hard link ile symbolic link arasındaki farkları açıklayın. Hangi durumlarda symbolic link zorunludur?

Hard link: Doğrudan i-node'a işaret eder; aynı dosya sistemi içinde çalışır

Soft link: Yol adına işaret eder; farklı dosya sistemlerine ve dizinlere bağlanabilir

Symbolic link zorunlu durumlar:

- Farklı dosya sistemine bağlantı gerektiğinde
- Dizin bağlantı gerektiğinde (hard link dizinlere yasak)
- Orijinal dosya silindiğinde ne olacağı önemli değilse (dangling link riski kabul ediliyor)

S3. FAT dosya sistemi için bellek gereksinimleri neden sorundur? 200 GB disk ve 1 KB blok boyutu varsayarak hesaplayın.

200 GB disk / 1 KB blok = $200 \times 10^6 = 200,000,000$ blok

Her FAT girişi 4 byte (FAT32) $\rightarrow 200,000,000 \times 4 = 800,000,000$ byte = ~ 800 MB RAM

Bu kadar büyük bir tablo sürekli bellekte tutulmak zorundadır

Bu FAT'ın temel dezavantajıdır; büyük diskler için ölçeklenmez

S4. Unix'te fsck hangi blok tutarsızlıklarını tespit eder ve nasıl çözer? 4 olası durumu açıklayın.

(a) Tutarlı: Blok hem bir dosyada hem serbest listede $\rightarrow$ Sorun yok

(b) Kayıp blok: Hiçbir yerde yok $\rightarrow$ Serbest listeye ekle

(c) Serbest listede çift: Blok iki kez listede → Listeyi yeniden oluştur

(d) Çift veri bloğu: İki dosya aynı bloğu paylaşıyor → Bloğu kopyala, her dosyaya bir kopya ver

S5. Journaling neden basit 'yazma sıralaması' (write ordering)'dan daha güçlüdür?

Yazma sıralaması: Yazmaların belli bir sırada yapılmasını garanti eder

- Çökmeden önce yarım kalan bir işlem belirsiz bırakılabilir

Journaling: Değişiklikler önce journal'a yazılır; atomic commit sağlar

- Çökmeden sonra journal replay ile tutarlı duruma geri dönülür

- Ek olarak: journaling 'intent log' tuttuğundan neyin tamamlanıp tamamlanmadığı net

Modern diskler yazma önbellekleri nedeniyle sadece sıralamaya güvenilemiyor

S6. 4 aşamalı mantıksal dump algoritmasını açıklayın. Neden değiştirilmiş dosyaların üst dizinleri de yedeklenmek zorundadır?

Aşama 1: Tüm değiştirilmiş dosyaları VE her dizini işaretle

Aşama 2: Altında değiştirilmiş dosya olmayan dizinlerin işaretini kaldır

Aşama 3: İşaretli dizinleri i-node sırasıyla dök

Aşama 4: İşaretli dosyaları dök

Neden üst dizinler?: Geri yüklemede dosyayı doğru yere koyabilmek için dizin yapısı gerekli

Dizin yapısı olmadan dosya geri yüklense bile adres bilinemez

S7. Cylinder group (silindir grubu) nedir ve performansa nasıl katkı sağlar?

Cylinder group: Disk belirli bölgelere ayrılır

Her grup kendi i-node listesi, serbest listesi ve veri alanına sahiptir

Performans katkısı: Referans yerselliği (locality of reference)

- Yeni dosyalar aynı grup içinde oluşturulur → disk kolu hareketi azalır

- Klasik Unix'te i-list başta, veri sonda → her erişimde uzun seek

- Cylinder group bu sorunu çözer: metadata ve veri aynı bölgede

S8. Açık (open) bir dosya unlink() ile silinirse ne olur? Sistem çökmesi bu senaryoda nasıl bir risk oluşturur?

unlink() ile link count azaltılır, sıfıra düşer ama ref count > 0 ise dosya gerçekten silinmez

Son close() çağrısıyla ref count = 0 olunca dosya gerçekten silinir

Çökme riski: Açık unlinked dosyayla sistem çökerse:

- Link count = 0, ref count = 0, ama bloklar hâlâ tahsisli

- Dosya sistemi bu i-node'a erişemez → kayıp blok durumu oluşur
 - Çözüm: fsck bu 'orphan' i-node'ları bulup blokları serbest bırakır
-

18. Kitap Bölüm Sonu Soruları ve Çözümleri

TB-1. Tanenbaum, Problem 4.1: Bir dosya sistemi 1 KB boyutunda bloklar kullanıyor. Küçük bir C programı 1 KB'dan az yer kaplıyor. Bu program derlenip yürütülebilir bir dosya oluşturduğunda toplam ne kadar disk bloğu gereklidir? Tüm olası durumları tartışın.

Kaynak dosya (.c): 1 blok (< 1 KB)

Yürütülebilir dosya (.out / a.out):

- Minimum: 1 blok (çok küçükse)
- Tipik: Başlık (header) + metin (text) + veri segmentleri nedeniyle birkaç blok

I-node blokları: Her dosya için 1 i-node gerekir

Toplam: En az 2 blok (kaynak + yürütülebilir); tipik 4-6 blok (i-node'lar dahil)

Not: İç parçalanma (internal fragmentation) nedeniyle son blok tam dolmayabilir

TB-2. Tanenbaum, Problem 4.6: Contiguous, linked ve indexed (i-node) tahsis yöntemlerini sıralı erişim ve rastgele erişim açısından karşılaştırın.

Sıralı erişim:

- Contiguous: En hızlı — bloklar bitişik, disk kolu hareketi az
- Linked: Orta — her blok sonrakini gösterse de fiziksel parçalanma olabilir
- I-node: İyi — blok adresleri bilinir, sıralı okuma yapılabilir

Rastgele erişim:

- Contiguous: İyi — başlangıç + offset ile blok bulunur
- Linked: Çok kötü — n. bloğa ulaşmak için n adım gerekir
- I-node: İyi — blok adresi i-node'dan doğrudan okunur (indirect için 1-3 ekstra okuma)

Sonuç: I-node her iki durumda da dengeli performans sunar

TB-3. Tanenbaum, Problem 4.10: 1 KB blok boyutlu bir i-node yapısı için 10 direct, 1 single indirect, 1 double indirect ve 1 triple indirect blok adresi vardır. Disk blok adresi 3 byte ise maksimum dosya boyutunu hesaplayın.

Pointer sayısı per blok: $1024 / 3 = 341$ (tam bölünmez $\rightarrow 341$)

Direct: $10 \times 1024 = 10,240$ byte

Single indirect: $341 \times 1024 = 349,184$ byte

Double indirect: $341^2 \times 1024 = 341 \times 349,184 = 119,062,144$ byte ≈ 113 MB

Triple indirect: $341^3 \times 1024 = 341 \times 119,062,144 \approx 40,598,791,168$ byte ≈ 37.8 GB

Toplam ≈ 37.8 GB + 113 MB + 341 KB + 10 KB ≈ 37.8 GB

TB-4. Tanenbaum, Problem 4.14: Mantıksal dump algoritmasında, değiştirilmemiş bir dizinin altında değiştirilmiş dosyalar varsa bu dizin yedeklenmek zorunda mıdır? Neden?

EVET, yedeklenmek zorundadır.

Sebebi: Geri yüklemede değiştirilmiş dosya yeni bir lokasyona kopyalanabilir

Bu durumda dosyanın tam yol adresini bulmak için tüm üst dizinlerin yapısı gereklidir

Dizin yedeklenmeseydi, geri yüklemede dosyanın nereye yerleştirileceği bilinemez

Algoritmanın Aşama 1-2 mantığı: Önce hepsini işaretleyip, sonra boşları eleme

Bu sayede yol bütünlüğü korunur

TB-5. Tanenbaum, Problem 4.20: Unix'te /etc/passwd ve /tmp/xyz dosyaları hard link olarak birbirine bağlanmak isteniyor. Bu mümkün müdür? Neden?

/etc genellikle kök dosya sisteminde (/), /tmp ise ayrı bağlı olabilir

Hard link farklı dosya sistemleri arasında KURULAMAZ

Sebebi: Hard link i-node numarasını kullanır; i-node numaraları dosya sistemine özeldir

Farklı dosya sistemlerinde aynı i-node numarası farklı dosyaları gösterir

Çözüm: Symbolic link kullanılabilir — yol adına işaret ettiğinden FS bağımsızdır

In -s /etc/passwd /tmp/xyz # symbolic link — çalışır

In /etc/passwd /tmp/xyz # hard link — çalışmaz (farklı FS ise)

TB-6. Tanenbaum, Problem 4.23: Bir dosya sistemi CD-ROM için tasarlanıyor. Hangi özellikler ISO 9660 temel standardında desteklenmez? Rock Ridge ve Joliet uzantıları hangi eksiklikleri giderir?

ISO 9660 temel sınırlamaları:

- Dosya adları 8.3 formatıyla sınırlı (büyük harf)
- Maksimum 8 seviye dizin derinliği
- POSIX öznitelikleri (izinler, owner) yok
- Symbolic link yok
- Unicode yok

Rock Ridge uzantıları (Unix için):

- POSIX öznitelikleri (PX), symbolic link (SL), uzun ad (NM), zaman damgaları (TF)

Joliet uzantıları (Windows için):

- Unicode, uzun ad (255 karakter), 8+ seviye dizin, uzantılı dizin adları

Hızlı Başvuru — Dosya Sistemleri Özeti

Konu	Anahtar Nokta
Dosya yapıları	Byte dizisi (modern), kayıt dizisi (eski), ağaç (DB)
Ardışık tahsis	Basit + hızlı; dış parçalanma sorunu
Linked tahsis	Esnek; rastgele erişim yavaş
FAT	Bağlı liste bellekte; büyük disk için RAM sorunu
I-node	10 direct + indirect; büyük dosya için çok katmanlı
Max dosya boyutu (4KB blok, 4B ptr)	~4 GB (double indirect)
FAT 200GB 1KB blok	~800 MB RAM gerekir
Hard link vs soft link	Hard: aynı FS, i-node; Soft: farklı FS, yol
Journaling	Değişiklik önce journal'a; atomic commit
fsck 4 durum	Tutarlı / Kayıp blok / Çift serbest / Çift veri
Logical dump 4 aşama	İşaretle → Eleme → Dizin dök → Dosya dök
Cylinder group	I-node + veri aynı bölgede; seek azaltır
VFS	Farklı FS tiplerini tek arayüzde gizler
ISO 9660 uzantıları	Rock Ridge (Unix/POSIX), Joliet (Windows/Unicode)

Dosya Sistemleri konusu anahtar noktalar